Project Navi

Bachelor Designer & Developer

NOAILLES Thomas

Project Management

Agile Methodology

Choice of methodology

For the development of Naviko, an **Agile approach based on the Kanban methodology** has been selected.

The project is developed by a single developer. In this context, the main challenge is not to coordinate a large development team, but to **organize tasks efficiently, maintain visibility on the project's progress, and adapt priorities when requirements or technical constraints evolve**.

Kanban is therefore suitable because it focuses on the continuous flow of work and visual management of tasks. It allows the project to remain flexible without introducing unnecessary organizational overhead.

Comparison with other approaches

Several project management approaches could have been considered.

A traditional **Waterfall approach** would require most requirements to be defined before development begins and would make changes later in the project more difficult and costly. This is not ideal for Naviko, where design decisions, technical constraints and testing results may lead to adjustments during development.

Scrum could also be used to manage the project through fixed sprints, a product backlog and regular ceremonies. However, Scrum is primarily designed to facilitate collaboration within a

development team. For a project mainly developed by one person, its roles, ceremonies and sprint structure would introduce additional organizational complexity without providing enough benefit.

Kanban is more appropriate for Naviko because it allows tasks to be managed continuously according to their priority and current status. It provides the structure needed to organize the project while remaining lightweight and flexible.

Problems addressed by Kanban

The main problems for using Kanban are:

- Lack of visibility: the Trello board provides a clear overview of planned, ongoing and completed work.
- Poor task prioritization: the Backlog allows future tasks to be collected and prioritized before they are started.
- Too many tasks being handled simultaneously: the workflow encourages limiting work in progress and focusing on the most important task first.
- Changing requirements: tasks can be reprioritized without having to reorganize a fixed sprint.
- Unclear project progress: moving cards through the workflow provides a simple visual representation of project progress.
- Incomplete tasks being considered finished: the Definition of Ready and Definition of Done provide common criteria for starting and completing work.

Implementation in Trello

Kanban is implemented through a Trello board representing the project's workflow:

Backlog > To Do > In Progress > Testing > Completed

The **Backlog** contains future work, including potential V2 features such as database integration or GPS navigation. Tasks are moved to **To Do** when they are sufficiently prepared according to the Definition of Ready.

Once development begins, the task moves to **In Progress**. After implementation, it enters **Test**, where the functionality is verified against its Acceptance Criteria and the project's quality requirements.

A task is moved to **Completed** only when it satisfies the Definition of Done.

This workflow provides Naviko with a lightweight project management framework while maintaining visibility, prioritization and control over the work in progress.

Trello Board

Trello is used as the main project management tool for Naviko and serves as the practical implementation of the Kanban methodology.

The board uses **User Stories** for user-facing functionality and dedicated technical or project-management cards for other activities. User Stories follow a consistent structure, including a clear user need, Acceptance Criteria and an appropriate checklist.

The board also includes dedicated **Definition of Ready (DoR)** and **Definition of Done (DoD)** cards, which provide common criteria for starting and completing work.

To maintain an efficient workflow, tasks are prioritized and work in progress is kept under control. This prevents too many tasks from being started simultaneously and makes the project's progress visible at all times.

The Trello board is therefore both a project tracking tool and a practical implementation of the Kanban principles defined in the previous section

Testing Strategy

Testing objectives

The testing strategy aims to ensure that Naviko meets its functional, technical, accessibility and quality requirements before being considered ready for release.

Testing will be used to verify that:

- the implemented features behave according to their requirements and Acceptance Criteria;
- the different components of the application interact correctly;
- the application remains stable and responsive during use;
- the user interface meets the defined accessibility requirements;
- the AI detection features provide sufficiently reliable results;
- the application remains usable for visually impaired users;
- modifications do not introduce regressions in existing features.

Because Naviko is designed to assist visually impaired users in outdoor environments, particular attention will be given to

detection reliability, accessibility and the consequences of incorrect or missing detections.

Testing methodology

Testing will be performed progressively throughout the development process rather than only at the end of the project.

Each feature will first be tested at the development level using automated tests where applicable. Once the individual components have been validated, integration and functional tests will verify that they work correctly together.

Before a feature can be considered complete, its Acceptance Criteria must be satisfied and the relevant tests must have been successfully completed.

Before a release, a broader validation phase will be performed, including regression, accessibility, performance, AI validation and user testing where applicable.

This approach makes it possible to identify defects early, reduce the cost of corrections and prevent incomplete or insufficiently tested features from being considered finished.

Types of testing

Several complementary types of testing will be used for Naviko.

Unit testing will verify individual functions or components in isolation. For example, a function responsible for processing detection results can be tested independently from the rest of the application.

Integration testing will verify that different components communicate correctly. An example for Naviko is the complete chain between the camera, the detection system and the alert mechanism.

Performance testing will evaluate aspects such as application responsiveness, detection processing time and the delay between a detection and the corresponding alert.

AI validation will specifically evaluate the reliability of the detection system using representative test data and different environmental conditions.

User testing will be used to validate the application with users, particularly visually impaired users, in order to evaluate usability, accessibility and the relevance of the provided feedback.

Testing tools

The exact tools will be defined during the technical implementation phase to ensure that each tool is appropriate for the corresponding technology and testing requirement.

Test validation and release decision

Testing results will be used to determine whether a feature can be considered complete.

A feature must satisfy its Acceptance Criteria, pass the relevant tests and meet the quality requirements defined for its level of criticality.

If a critical test fails, the feature will not be considered **Done**. The related task will return to development, the issue will be corrected and the relevant tests will be executed again.

This process ensures that the Definition of Done is supported by objective testing results rather than by the completion of development alone.

Production Readiness and Release Criteria

Production readiness criteria

Before releasing Naviko to production, the application must meet a set of minimum quality and validation criteria.

The following conditions are considered mandatory:

Functional requirements

- All V1 critical features must be implemented.
- All Acceptance Criteria associated with these features must be satisfied.

Automated testing

- All required automated tests must pass.
- No unresolved test failure affecting a critical feature can remain before release.

Accessibility and user validation

- The main accessibility requirements must be validated.
- User testing must confirm that the main interaction mechanisms are understandable and usable.

AI detection reliability

- The main detection features must be validated using the defined AI testing methodology.
- Detection results must meet the quality thresholds established during validation.
- Critical detection issues must be resolved before release.

These criteria ensure that a feature is not considered ready for production simply because its development is complete. It must also satisfy the project's functional, accessibility and reliability requirements.

Release decision and blocking issues

A release decision will be based on the results of the validation process.

The following issues are considered **release blockers**:

- **Critical feature incomplete:** an essential V1 feature has not been implemented or does not satisfy its Acceptance Criteria.
- **Critical accessibility issue:** an accessibility problem prevents or significantly limits the effective use of an essential feature.
- **Critical AI detection reliability issue:** a critical detection feature does not meet the defined reliability requirements or produces results that could negatively affect the safe use of the application.

When a release blocker is identified, the release is considered **No-Go**. The related task remains outside the **Done** column and is returned to development. The issue must then be corrected and the relevant tests repeated before a new release decision is made.

Non-critical issues that do not affect essential functionality, accessibility or detection reliability may be documented and scheduled for a future version, provided that they do not represent an unacceptable risk.

This approach ensures that production deployment is based on defined quality and risk criteria rather than simply on the completion of the development schedule.

Release Blocking & Decision Strategy

The release decision is based on the production readiness criteria defined in the previous section. The objective is to

ensure that no critical issue is knowingly introduced into production.

Release decision

Before each release, the relevant tests and validation activities are reviewed.

The decision follows a simple **Go / No-Go** approach:

- **Go:** all mandatory criteria are satisfied and no release blocker remains.
- **No-Go:** at least one critical release blocker has been identified.

A release blocker can be:

- an incomplete critical V1 feature;
- a critical accessibility issue;
- a critical AI detection reliability issue;
- a failed automated test affecting a critical feature.

Handling a release blocker

When a release blocker is identified, the affected feature is not considered **Done**.

The issue is documented and a corrective task is created or updated in the Trello board. The task is then returned to the development workflow.

The release can only proceed once the blocker has been resolved and the relevant validation criteria have been successfully completed.

For example, if testing shows that a critical detection feature does not reach its required reliability level, the release is blocked. The detection feature is returned to development, the problem is addressed and the relevant AI validation tests are performed again before a new release decision.

Non-critical issues

Not every issue necessarily blocks a release.

A minor issue that does not affect critical functionality, accessibility or detection reliability can be documented and added to the Backlog for a future release.

This allows the project to maintain a balance between **quality requirements and delivery constraints**, while ensuring that issues capable of significantly affecting the use of Naviko are not postponed.

Risk Management Approaches

Risk management is used throughout the development of Naviko to identify potential problems early, reduce their impact and define appropriate actions if they occur.

The risk management approach is based on three complementary strategies: **prevention, mitigation and contingency planning**.

Risk prevention

Prevention aims to reduce the probability of a risk occurring by identifying and addressing potential problems before they affect the project.

For Naviko, prevention includes:

- identifying technical and functional risks before development;
- validating important requirements before implementation;
- testing critical features progressively during development;
- validating AI detection under different environmental conditions;
- identifying accessibility issues early through design and testing;

For example, the reliability of AI detection can be evaluated using representative test cases before the feature is considered ready for release. This helps identify weaknesses before they become production issues.

Risk mitigation

Some risks cannot be completely eliminated. Mitigation therefore aims to reduce their probability or potential impact.

For example, AI detection may become less reliable in poor lighting or when an object is partially obstructed. To reduce the impact of uncertain detections, the application can use appropriate confidence thresholds and avoid generating an alert when the system cannot provide a sufficiently reliable result.

Other mitigation measures include prioritizing critical features, limiting work in progress and performing regular testing throughout development.

Risk mitigation is therefore closely connected to the project's testing strategy and release criteria.

Contingency and corrective actions

Contingency planning defines what should be done if a risk occurs despite preventive and mitigation measures.

When a significant problem is identified, its impact and criticality are evaluated. If the problem affects a critical feature, accessibility or AI detection reliability, the release may be blocked according to the project's Release Blocking & Decision Strategy.

For example, if testing shows that a critical detection feature does not reach the required reliability level, the release is blocked. A corrective task is created in Trello, the feature is returned to development and the relevant tests are performed again before a new release decision is made.

This approach ensures that risks are not only identified, but actively managed throughout the project lifecycle.